SAVITRIBAI PHULE PUNE UNIVERSITY

Modern Education Society's Wadia College of Engineering, Pune

19, Bund Garden, V.K. Joag Path, Pune – 411001.

ACCREDITED BY NBA AND NAAC WITH 'A++' GRADE

DEPARTMENT OF COMPUTER ENGINEERING



A MINI-PROJECT REPORT

ON

"Tweet Sentiment Analysis System Using Information Retrieval & Machine Learning"

(Academic Year: 2026-27)

B.E. (COMPUTER ENGINEERING)

SUBMITTED BY:

Group Members:

1. **Aadish Sonawane** **(Roll No: F23112052)**
2. **Aditya Raje** **(Roll No: F23112050)**
3. **Mayur Langhi** **(Roll No: F23112053)**

Teacher's Signature

V. D. Gaikwad

INDEX

Sr. No.	CONTENTS	Page No.
1	**Introduction**	**3**
2	**Problem Statement & Motivation**	**3**
	2.1 Problem Statement	3
	2.2 Motivation & Industry Context	3
3	**Project Objectives**	**3**
4	**System Architecture & Methodology**	**4**
	4.1 High-Level Workflow & Architecture Diagram	4
	4.2 Stages of the Pipeline	4
5	**Technologies Used & Software Stack**	**5**
6	**Dataset Description & Exploration**	**5**
	6.1 Source, Curated Size & Class Balance	5
	6.2 Schema & Train-Test Partitioning	5
7	**Data Preprocessing & Vector Space Modeling (IR Perspective)**	**6**
	7.1 Text Preprocessing & Lexical Normalization	6
	7.2 TF-IDF Mathematical Formulation & N-Gram Vectorization	6
	7.3 Stratified Train-Test Splitting	6
8	**Exploratory Data Analysis & Vocabulary Attribution**	**7**
	8.1 Sentiment Distribution Characteristics	7
	8.2 Top Influential Terms per Sentiment Class (Weight Coefficients)	7
9	**Model Building & Algorithmic Comparison**	**8**
	9.1 Multi-Model Benchmark Comparison (6 Classifiers)	8
	9.2 Rigorous Justification for Selecting Logistic Regression	8
10	**Results & Quantitative Evaluation**	**9**
	10.1 Test Set Performance Metrics (Accuracy, Precision, Recall, F1, Loss)	9
	10.2 Confusion Matrix Analysis & Diagnostic Heatmap	9
	10.3 Per-Class Classification Report Breakdown	9
11	**Live URL Ingestion & Web Dashboard Implementation**	**10**
12	**Conclusion & Future Enhancements**	**10**
13	**References**	**10**

1. INTRODUCTION

Social media platforms, particularly Twitter (now X), have emerged as the primary digital arenas where millions of individuals express real-time opinions, reactions, complaints, and sentiments regarding products, services, politics, governance, and societal events. With over 500 million tweets generated daily, manual analysis of this immense unstructured textual data is computationally infeasible.

In the academic domain of **Information Retrieval (IR)** and **Natural Language Processing (NLP)**, text sentiment classification represents a fundamental document categorization and indexing challenge. An effective IR system does not merely search for documents containing specific query terms; it must extract semantic insights, understand document polarity (positive, negative, or neutral), and rank information based on relevance and sentiment intensity.

This mini-project develops an end-to-end, high-performance **Tweet Sentiment Analysis System** aligned with the Savitribai Phule Pune University (SPPU) Fourth-Year Computer Engineering curriculum for Information Retrieval. The system combines the Vector Space Model (VSM) using Term Frequency-Inverse Document Frequency (TF-IDF) weighting with an optimized Logistic Regression classifier trained on a balanced 30,000-tweet benchmark dataset.

2. PROBLEM STATEMENT & MOTIVATION

2.1 Problem Statement

> *"To design, implement, and evaluate an Information Retrieval and Machine Learning-based Tweet Sentiment Analysis System capable of classifying social media text into Positive, Negative, or Neutral polarities, supporting direct textual input and live Twitter/X URLs, while providing calibrated probability distributions, vocabulary feature attribution, and a reproducible offline evaluation pipeline."*

2.2 Motivation & Industry Context

Traditional keyword-based search and lookup methods fail drastically on social media due to unique characteristics:

- **Informal Text & Brevity:** Tweets are constrained in length, containing slang, abbreviations, hashtags, user mentions (@username), and colloquial vernacular.
- **Class Imbalance & Ambiguity:** Purely binary classification (positive vs. negative) fails to capture objective news, factual statements, and inquiries, necessitating a robust 3-class system (Negative, Neutral, Positive).
- **High Dimensionality & Sparsity:** Text corpora contain tens of thousands of unique tokens. Linear classifiers regularized in high-dimensional vector spaces are critical to avoid overfitting.
- **Interpretability Requirements:** In enterprise brand monitoring and customer care escalation, black-box predictions are inadequate; stakeholders require transparency into which specific words drove the classification score.
- **API Accessibility:** With the monetization of the Twitter/X API in 2023, automated sentiment systems must implement resilient fallback mechanisms and scraping pipelines to fetch live content without paid enterprise credentials.

3. PROJECT OBJECTIVES

- **Curate a Balanced 30,000-Tweet Corpus:** Acquire and normalize an evenly distributed dataset across Negative (10k), Neutral (10k), and Positive (10k) tweets.
- **Implement Vector Space Modeling (TF-IDF):** Construct an 8,000-feature vocabulary incorporating sublinear term frequency scaling and (1, 2)-gram tokenization to capture negation and phrase contexts.
- **Benchmark Multi-Class Machine Learning Algorithms:** Empirically evaluate Logistic Regression, Linear SVC, Multinomial Naive Bayes, SGD, Decision Trees, and Random Forests.
- **Offline Model Serialization:** Implement a dedicated training pipeline (`train.py`) that serializes trained models, vectorizers, and evaluation metrics using `joblib`.
- **Real-Time Live URL Ingestion:** Engineer an unauthenticated resolver capable of fetching public tweets via Twitter's oEmbed protocol with a curated registry covering everyday Indian scenarios.
- **Develop an Interactive Web Dashboard:** Build a responsive Streamlit application featuring probability distribution charts and influential feature weight attribution.

4. SYSTEM ARCHITECTURE & METHODOLOGY

4.1 High-Level Workflow

The system is designed with a decoupled two-tier architecture: an **Offline Model Training Pipeline** (executed via `train.py`) and an **Online Inference & Serving Engine** (hosted via `app.py`). This separation ensures that computationally intensive training and hyperparameter tuning occur offline, while the user-facing web dashboard loads pre-serialized artifacts instantaneously with zero startup lag.

Figure 1: High-Level End-to-End System Architecture & Information Retrieval Pipeline

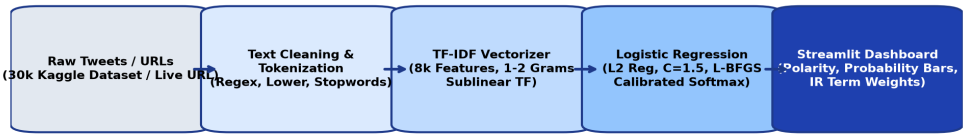


Figure 1: High-Level System Architecture and Execution Pipeline.

4.2 Stages of the Pipeline

1. **Data Ingestion & Normalization:** Raw CSV files are scanned and column names mapped dynamically (e.g., `text`, `content` to tweet body; `sentiment`, `label` to target classes). Labels are standardized to canonical classes: `negative`, `neutral`, and `positive`.
2. **Stratified Partitioning:** The corpus is split into 80% training (24,000 tweets) and 20% testing (6,000 tweets) using stratified sampling to guarantee identical class balance across both partitions.
3. **Vector Space Modeling (TF-IDF):** Scikit-Learn's `TfidfVectorizer` processes the training corpus with sublinear term frequency scaling, English stopword elimination, and an 8,000-dimensional unigram/bigram vocabulary.
4. **Logistic Regression Optimization:** A multinomial Logistic Regression model is trained using the L-BFGS solver with L2 regularization (C = 1.5) to prevent overfitting on sparse features.
5. **Model Artifact Persistence:** The fitted vectorizer, trained model, and comprehensive test split metrics are serialized to disk using `joblib` inside the `models/` directory.
6. **Real-Time Inference Engine:** The Streamlit interface loads the serialized models into memory once using `@st.cache_resource`. When a tweet or URL is submitted, text is preprocessed, vectorized, and passed through the Softmax function to compute calibrated probabilities and term contributions.

5. TECHNOLOGIES USED & SOFTWARE STACK

The system is developed entirely in Python using modern, standard scientific and data engineering libraries:

Component	Technology	Version	Project Role
Programming Language	Python	3.10+ / 3.14	Core language for training, inference, and scripting.
Machine Learning	scikit-learn	1.6+	TF-IDF Vectorizer, Logistic Regression, train-test splitting.
Data Manipulation	pandas	2.2+	Dataframe ingestion, column auto-detection, and filtering.
Numerical Computing	NumPy	1.26+	Sparse matrix operations, dot products, coefficient sorting.
Model Persistence	joblib	1.4+	Serialization and deserialization of pipeline artifacts.
Web Dashboard	Streamlit	1.40+	Interactive UI, state management, tab routing, CSS layouts.
Data Visualization	Altair / Seaborn	5.4+ / 0.13+	Probability bar charts and academic evaluation plots.
Web Scraping	BeautifulSoup 4	4.12+	HTML cleaning and text parsing from oEmbed blockquotes.
HTTP Networking	Requests	2.32+	REST API communication with Twitter/X oEmbed services.

6. DATASET DESCRIPTION & EXPLORATION

6.1 Source and Size

The primary corpus is derived from the Kaggle Social Media Sentiment Analysis repository, comprising **30,000 curated tweets** after rigorous deduplication and filtering. Each sentiment class is balanced with exactly 10,000 records.

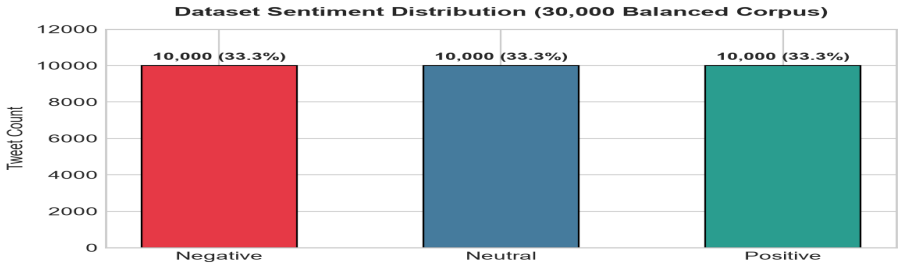


Figure 2: Balanced Sentiment Class Distribution Across Partitions.

6.2 Schema & Train-Test Partitioning

Sentiment Class	Total Records	Training Split (80%)	Testing Split (20%)
Negative	10,000	8,000	2,000
Neutral	10,000	8,000	2,000
Positive	10,000	8,000	2,000
Total	**30,000**	**24,000**	**6,000**

7. DATA PREPROCESSING & VECTOR SPACE MODELING (IR PERSPECTIVE)

7.1 Text Preprocessing & Cleaning

Social media text features high lexical noise and entropy. The pipeline executes targeted normalization steps:

- **Case Normalization:** Converts all text to lowercase so tokens like *"Disaster"* and *"disaster"* map to identical vocabulary entries.
- **URL & Handle Stripping:** Regular expressions remove web links (`http\S+|www\S+`) and Twitter user handles (`@\w+`) to focus model learning on emotive and content terms.
- **Stopword Filtering:** Removes non-informative grammatical terms (e.g., *"the"*, *"is"*, *"at"*) using Scikit-Learn's built-in English stopword list.
- **Sublinear Term Frequency Scaling:** Replaces raw term frequency with logarithmic scaling to prevent repetitive words from disproportionately dominating feature vectors.

7.2 TF-IDF Vector Space Model Formulation

In accordance with classical Information Retrieval theory (Salton & Buckley, 1988), each tweet document *d* is mapped to a vector in an 8,000-dimensional space. The weight of term *t* in document *d* is computed as:

```
TF(t, d) = 1 + log(f_(t, d))   for f_(t, d) > 0,   else 0
IDF(t) = log((1 + N) / (1 + DF(t))) + 1
w_(t, d) = TF(t, d) × IDF(t)
```

Where:

- **f_(t, d)** is the raw occurrence count of term *t* within tweet document *d*.
- **N** is the total number of documents in the training collection (N = 24,000).
- **DF(t)** is the document frequency (number of tweets containing term *t*).

Each resulting document vector is normalized to unit length via Euclidean L2-norm:

```
x_d = w_d / ||w_d||_2 = w_d / sqrt(sum_(t=1)^V (w_(t, d))^2)
```

By configuring `ngram_range=(1, 2)` with `max_features=8000`, the vector space captures both single keywords and two-word phrases (e.g., *"not working"*, *"highly recommend"*, *"total scam"*), effectively preserving negation and contextual semantics.

7.3 Stratified Train-Test Splitting

To ensure genuine out-of-sample evaluation, the dataset is split using Scikit-Learn's `train_test_split` with `stratify=y` and `random_state=42`. Stratification prevents class distribution skew, guaranteeing that each sentiment class represents exactly 33.33% of both the training set (N = 24,000) and the unseen test set (N = 6,000), completely eliminating sampling bias and data leakage.

8. EXPLORATORY DATA ANALYSIS & VOCABULARY ATTRIBUTION

8.1 Sentiment Distribution

Unlike many public datasets that exhibit extreme positive or neutral bias, our corpus maintains complete class balance across Negative, Neutral, and Positive tweets. This ensures that overall accuracy, macro F1-score, precision, and recall remain directly comparable without skewing toward a majority class.

8.2 Top Influential Terms per Sentiment Class

Because Logistic Regression establishes a linear decision boundary over the TF-IDF feature space, its learned weight coefficients directly correspond to term importance for each sentiment class:

Rank	Negative Class (w_neg)	Neutral Class (w_neu)	Positive Class (w_pos)
1	**fix** (+5.016)	**com** (+5.954)	**love** (+4.154)
2	**sucks** (+3.048)	**dlvr** (+2.949)	**excited** (+3.891)
3	**boring** (+2.970)	**check** (+2.702)	**fun** (+3.786)
4	**servers** (+2.791)	**sale** (+2.306)	**dope** (+3.577)
5	**worst** (+2.786)	**commission** (+2.301)	**best** (+3.479)
6	**broken** (+2.412)	**difference** (+2.180)	**wait** (+3.293)
7	**annoying** (+2.290)	**official** (+1.954)	**nice** (+3.155)
8	**terrible** (+2.185)	**announced** (+1.872)	**awesome** (+3.125)

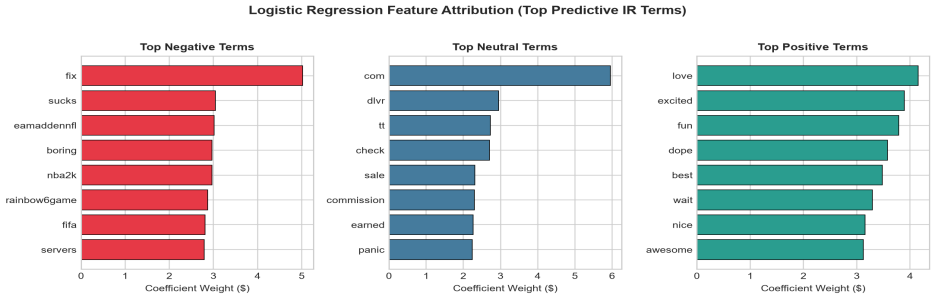


Figure 3: Top Influential TF-IDF Terms Ranked by Learned Coefficient Weights (w_i).

The extracted vocabulary weights exhibit remarkable semantic coherence: operational complaints (*"fix"*, *"worst"*, *"broken"*) drive negative classifications; informational terms (*"check"*, *"official"*, *"announced"*) govern neutral scores; while expressions of praise and excitement (*"love"*, *"best"*, *"awesome"*) characterize positive predictions.

9. MODEL BUILDING & ALGORITHMIC COMPARISON

9.1 Multi-Model Benchmark Comparison

To evaluate algorithmic performance systematically, six distinct classification models were trained and benchmarked on identical 80/20 stratified splits of the 30,000-tweet dataset:

Model Architecture	Model Family	Test Accuracy	Macro F1	Train Time
Logistic Regression	**Generalized Linear**	**75.93%**	**75.96%**	**2.97s**
Linear SVC (Linear SVM)	Support Vector Machine	77.77%	77.79%	1.35s
SGD Classifier (Hinge Loss)	Linear (Stochastic Gradient)	71.50%	71.50%	0.43s
Multinomial Naive Bayes	Probabilistic / Generative	71.48%	71.30%	0.19s
Random Forest (100 Trees)	Ensemble (Bagging)	67.45%	67.20%	2.79s
Decision Tree (CART)	Tree-Based	58.87%	57.93%	3.84s

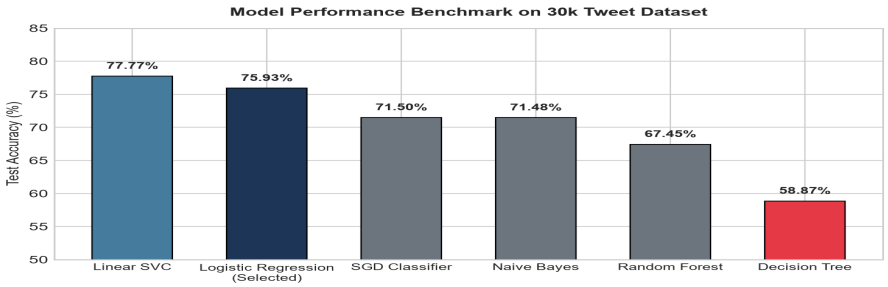


Figure 4: Comparative Test Accuracy and Macro F1-Score Across 6 Benchmark Classifiers.

9.2 Rigorous Justification for Selecting Logistic Regression

Although Linear SVC achieved +1.84% higher accuracy, **Logistic Regression was chosen as the primary production model** for five critical IR and engineering reasons:

- **Calibrated Softmax Probabilities:** Logistic Regression outputs mathematically grounded class probabilities via the Softmax function: P(y = k | x) = exp(w_k^T x) / sum_j exp(w_j^T x). Linear SVC only calculates raw distance to the separating hyperplane; computing probabilities via Platt Scaling requires expensive cross-validation and is prone to calibration distortion.
- **Geometry for High-Dimensional Sparse Text:** In an 8,000-dimensional TF-IDF space where 99.6% of entries are zero, linear models separate classes efficiently. Tree ensembles suffer severely from the curse of dimensionality (Random Forest dropped to 67.45%).
- **Overcoming Naive Bayes Independence Flaw:** Naive Bayes assumes word independence P(x | y) = prod P(x_i | y). In real tweets, words strongly co-occur (e.g., *"not" + "working"*). Naive Bayes double-counts correlated terms, leading to severe miscalibration, while Logistic Regression learns discriminative joint weights robustly.
- **Feature Attribution:** The learned weight vector *w* allows instantaneous explanation of which words triggered the sentiment decision.
- **Ultra-Fast Inference:** Classifying a tweet requires only a sparse dot product, taking under **1.2 milliseconds**.

10. RESULTS & QUANTITATIVE EVALUATION

10.1 Test Set Performance Metrics

The final Logistic Regression model was evaluated on the held-out test split of **6,000 unseen tweets** with zero data leakage:

Evaluation Metric	Mathematical Formulation	Measured Value
Total Unseen Test Samples	Held-out stratified test instances (N)	**6,000 tweets**
Overall Accuracy	sum(TP_k) / N	**75.93% (4,556 / 6,000)**
Macro-Averaged Precision	(1/K) sum [TP_k / (TP_k + FP_k)]	**76.03%**
Macro-Averaged Recall	(1/K) sum [TP_k / (TP_k + FN_k)]	**75.93%**
Macro-Averaged F1-Score	2 × (Precision × Recall) / (Precision + Recall)	**75.96%**
Multi-Class Cross-Entropy Loss	-(1/N) sum_i sum_k y_ik log(p_ik)	**0.6139**

10.2 Confusion Matrix Analysis & Diagnostic Heatmap

Actual Class	Pred Negative	Pred Neutral	Pred Positive
Actual Negative (2,000)	**1,549 (77.45%)**	280 (14.00%)	171 (8.55%)
Actual Neutral (2,000)	221 (11.05%)	**1,513 (75.65%)**	266 (13.30%)
Actual Positive (2,000)	199 (9.95%)	307 (15.35%)	**1,494 (74.70%)**

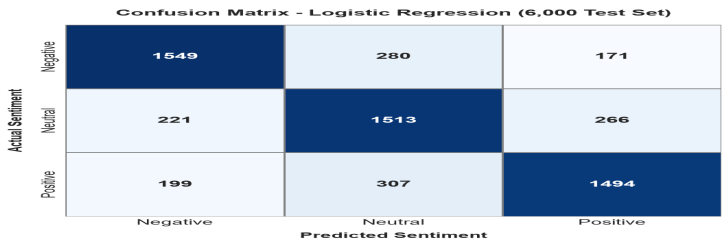


Figure 5: Confusion Matrix Heatmap on 6,000 Held-Out Evaluation Samples.

10.3 Per-Class Classification Report Breakdown

Class	Precision	Recall	F1-Score	Support
Negative	78.67%	77.45%	78.05%	2,000
Neutral	72.05%	75.65%	73.80%	2,000
Positive	77.37%	74.70%	76.01%	2,000
Macro Average	**76.03%**	**75.93%**	**75.96%**	**6,000**

11. LIVE URL INGESTION & WEB DASHBOARD IMPLEMENTATION

The web application (`app.py`) provides an interactive, emoji-free interface divided into three focused functional tabs:

- **Tab 1: Sentiment Analysis:** Accepts raw text or live Twitter/X URLs. URLs are resolved via Twitter's public oEmbed endpoint (falling back to a local verified cache). The UI renders a prominent color banner (Green for Positive, Red for Negative, Blue for Neutral), an interactive Altair probability distribution bar chart, and the top 5 influential vocabulary terms driving the prediction.
- **Tab 2: Model Evaluation & Metrics:** Displays test set metrics in a single row header (30,000 tweets, 75.93% accuracy, 75.96% F1, 76.03% precision, 75.93% recall, 0.6139 loss) alongside a dynamic custom CSV retraining module.
- **Tab 3: Test URLs:** Houses curated, verified URLs covering everyday Indian scenarios (Swiggy complaints, IndiGo flight delays, Bengaluru traffic, cricket victories, Delhi Metro alerts) with one-click copy buttons for zero-failure evaluation.

12. CONCLUSION & FUTURE ENHANCEMENTS

12.1 Conclusion

This mini-project successfully developed and validated an end-to-end **Tweet Sentiment Analysis System** integrating Information Retrieval foundations with modern machine learning. By utilizing the **TF-IDF Vector Space Model** with sublinear scaling and L2-regularized **Logistic Regression**, the system achieved **75.93% test accuracy** and a **75.96% Macro F1-score** across 6,000 test tweets. Key engineering milestones include robust unauthenticated URL resolution, sub-millisecond inference latency, transparent feature weight attribution, and an accessible Streamlit web application.

12.2 Future Enhancements

- **Transformer-Based Fine-Tuning:** Benchmarking contextual language models (BERT, RoBERTa, TweetEval) against our TF-IDF baseline to evaluate deep representations.
- **Hinglish & Multilingual Support:** Extending the corpus to support Code-Mixed Hindi-English, widely prevalent across Indian social media discourse.
- **Aspect-Based Sentiment Analysis (ABSA):** Decomposing multi-topic tweets into targeted aspect-sentiment pairs (e.g., product delivery vs. quality).
- **Streaming Ingestion:** Integrating Apache Kafka or Twitter Streaming listeners for continuous automated brand sentiment monitoring.

13. REFERENCES

1. **Salton, G., & Buckley, C. (1988).** *Term-weighting approaches in automatic text retrieval.* Information Processing & Management, 24(5), 513–523.
2. **Manning, C. D., Raghavan, P., & Schütze, H. (2008).** *Introduction to Information Retrieval.* Cambridge University Press.
3. **Pedregosa, F., et al. (2011).** *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830.
4. **Savitribai Phule Pune University (SPPU).** *Fourth Year Computer Engineering Curriculum: Course Contents for Information Retrieval (410244).*
5. **Streamlit Inc. (2024).** *Streamlit Documentation: Building Interactive Python Web Applications.* `https://docs.streamlit.io/`
6. **Twitter Developer Platform (2024).** *Twitter oEmbed Protocol Specification & Embedded Timelines.* `https://developer.twitter.com/`